Slide 1: Introduction to OpenRewrite

Title: Automated Refactoring for the Modern Era: An Introduction to OpenRewrite

Subtitle: A powerful open-source tool for large-scale, automated code transformations.

Content:
This presentation will introduce OpenRewrite, an innovative tool designed to automate tedious and error-prone code refactoring. We will explore its core principles, understand its key components, and see how it can drastically simplify major technical debt efforts, especially Java and Spring Boot upgrades. By the end, you'll have a clear understanding of how to leverage OpenRewrite to modernize your codebase efficiently.

Slide 2: The Problem with Manual Upgrades

Title: The Challenge of Modernization: The Upgrade Treadmill

Content:
Software frameworks and libraries are constantly evolving. While these updates bring new features, performance improvements, and security fixes, the migration process can be a significant pain point.

- **Time-Consuming:** Manual refactoring for a large codebase can take weeks or even months.
- **Error-Prone:** Human error can introduce new bugs, leading to costly and time-consuming debugging cycles.
- **Loss of Context:** Developers often lose track of the original code's intent when making widespread changes, leading to subtle regressions.
- **Stagnation:** The high effort of upgrades can lead to teams avoiding them, resulting in accumulating technical debt and security vulnerabilities.

Slide 3: What is OpenRewrite?

Title: OpenRewrite: The Solution

Content:
OpenRewrite is an open-source automated refactoring tool that makes programmatic changes to code. It operates by manipulating a Lossless Semantic Tree (LST) of your code, a powerful representation that preserves both the code's structure and its original formatting, including whitespace and comments.
Key Features:

- **Automated:** It automates repetitive code changes, freeing up developers for more

complex tasks.
- **Type-Aware:** Unlike simple search-and-replace, OpenRewrite understands the code's semantic meaning and type information, ensuring accurate transformations.
- **Recipe-Driven:** It uses a collection of pre-defined "recipes" to perform specific, targeted refactorings.

Slide 4: OpenRewrite Architecture

Title: How It Works: Recipes, Visitors, and LSTs

Content:
OpenRewrite's power lies in its unique architecture.

1. **Source Code to LST:** OpenRewrite first parses your source code and creates a Lossless Semantic Tree (LST). The LST is a more advanced version of an Abstract Syntax Tree (AST), retaining all formatting information.
2. **Recipes:** Recipes are the "what." They are a series of instructions that describe the desired change. A recipe can be a single task or a composite of multiple sub-recipes.
3. **Visitors:** Visitors are the "how." They traverse the LST, identify code that matches a recipe's criteria, and apply the transformation.
4. **LST to Source Code:** After the visitor has modified the LST, OpenRewrite prints the new tree back to a human-readable source file, preserving the original formatting.

Slide 5: The OpenRewrite Recipe Ecosystem

Title: A Look Inside: The Recipes

Content:
Recipes are the heart of OpenRewrite. They are simple Java classes or YAML files that define a set of transformations. There are thousands of publicly available recipes for common tasks.

- **Declarative Recipes (YAML):** Simple and common. They are composed of other existing recipes and are often used to combine multiple tasks.
- **Imperative Recipes (Java):** More powerful and complex. They are written in Java and allow for fine-grained, conditional logic for transformations.
- **Refaster Templates:** A middle ground, perfect for straightforward one-to-one code replacements.

Slide 6: Common Use Cases

Title: More Than Just Upgrades: Diverse Applications

Content:

While it excels at migrations, OpenRewrite has a wide range of applications:

- **Framework Migrations:** Upgrading from old versions of frameworks like Spring, Micronaut, or Quarkus.
- **Dependency Updates:** Automatically upgrading dependencies to newer, more secure versions.
- **CVE Patching:** Applying security vulnerability patches at scale across a codebase.
- **API Migrations:** Changing method signatures or package names when a library updates.
- **Code Style & Best Practices:** Enforcing consistent coding conventions and applying static analysis fixes.

Slide 7: Pros of OpenRewrite: Efficiency & Accuracy

Title: The Benefits, Part I: Faster and More Reliable

Content:
OpenRewrite brings significant advantages to the software development lifecycle.

- **Dramatic Time Savings:** What would take a team of developers days or weeks can be completed in minutes.
- **Reduced Human Error:** Automation eliminates the risk of typos, subtle logical errors, and missed changes.
- **Consistency:** Every transformation is applied consistently, ensuring a uniform code style and avoiding fragmentation.
- **Deterministic Results:** The same recipe applied to the same code will always produce the same result, making it predictable and trustworthy.

Slide 8: Pros of OpenRewrite: Scale & Maintainability

Title: The Benefits, Part II: Scaling Modernization

Content:
OpenRewrite is built for enterprise-level challenges.

- **Refactoring at Scale:** It can be run across thousands of repositories simultaneously, allowing large organizations to tackle technical debt on a massive scale.
- **Seamless Integration:** Plugins for build tools like Maven and Gradle allow it to fit directly into your existing CI/CD pipelines.
- **Codebase as a Product:** By continuously modernizing, the codebase remains clean and easy to maintain, reducing long-term costs.
- **Knowledge Transfer:** Recipes can be shared across teams, standardizing modernization

efforts and capturing institutional knowledge.

Slide 9: Cons of OpenRewrite

Title: Limitations and Considerations

Content:
While incredibly powerful, OpenRewrite is not a magic bullet.

- **Contextual Complexity:** It cannot understand complex business logic or architectural decisions. It can only change what is programmatically defined.
- **Imperative Logic:** It cannot fix every problem. Changes that require human-level reasoning or a complete redesign of a component cannot be fully automated.
- **Recipe Maintenance:** As codebases and frameworks evolve, recipes themselves need to be maintained and updated.
- **Initial Setup:** There is an initial learning curve to understand the core concepts and configure the build plugins correctly.

Slide 10: The Java Upgrade Challenge

Title: Tackling the Java Migration Mountain

Content:
Upgrading Java versions is a common and necessary task.

- **API Removals:** APIs deprecated in one version are often removed in a subsequent one (e.g., javax.script in Java 11).
- **Module System:** The shift to the Java Platform Module System (JPMS) in Java 9 introduced significant changes.
- **Dependency Conflicts:** New Java versions may introduce incompatibilities with older dependencies that require updates.
- **Build Tool Configuration:** The pom.xml or build.gradle files need to be updated to target the new Java version.

Slide 11: The OpenRewrite Java Upgrade Solution

Title: Automating Java Upgrades with Recipes

Content:
OpenRewrite provides specific recipes to handle Java migrations. For example, org.openrewrite.java.migrate.UpgradeToJava17 is a composite recipe that performs a series of transformations:

- Updates build files (Maven/Gradle) to target Java 17.

- Removes deprecated APIs and replaces them with modern alternatives.
- Updates dependencies that are not compatible with Java 17.
- Converts old constructs to modern ones (e.g., String.format to String.formatted).

Slide 12: Step-by-Step: Preparing for a Java Upgrade

Title: The Process, Part I: Setup

Content:
Before running an upgrade recipe, ensure your project is ready.

1. **Backup:** Always back up your project or commit all pending changes to version control.
2. **Add Plugin:** Add the rewrite-maven-plugin or rewrite-gradle-plugin to your project's build file.
3. **Define Recipes:** Add the necessary recipe dependencies and configure the plugin to run the specific upgrade recipe you need (e.g., org.openrewrite.java.migrate.UpgradeToJava17).

Slide 13: Step-by-Step: Running the Java Upgrade

Title: The Process, Part II: Execution

Content:
Once configured, running the recipe is simple.

1. **Dry Run:** First, perform a dry run to see the proposed changes without applying them.
    - mvn rewrite:dryRun
    - This generates a report showing which files would be changed and what the changes are. It's a great way to inspect the results.
2. **Run:** If you are satisfied with the dry run, execute the run command.
    - mvn rewrite:run
    - This applies the changes directly to your source files.
3. **Review and Commit:** Review the applied changes using git diff, manually address any remaining issues, and commit the changes.

Slide 14: The Spring Boot Upgrade Challenge

Title: Migrating to Spring Boot 3.x

Content:
The migration from Spring Boot 2.x to 3.x is a major undertaking.

- **Jakarta EE:** The most significant change is the move from the javax to the jakarta namespace. This affects virtually every Spring application.
- **Java 17 Requirement:** Spring Boot 3.x requires a minimum of Java 17.
- **API Changes:** Many classes and methods have been deprecated or removed.
- **Configuration Properties:** Numerous application.properties and application.yml keys have been renamed.

Slide 15: The OpenRewrite Spring Boot Solution

Title: Simplifying the Spring Boot 3 Migration

Content:
OpenRewrite offers a comprehensive org.openrewrite.java.spring.boot3.UpgradeSpringBoot_3_0 recipe that automates this complex migration. This composite recipe includes:

- Migrating to Jakarta EE.
- Updating the build file for Java 17.
- Fixing dependency version numbers.
- Renaming deprecated configuration properties.
- Refactoring code to use new APIs.

Slide 16: Steps for a Spring Boot Upgrade

Title: A Practical Example: Running the Recipe

Content:
Let's look at the pom.xml configuration for a Spring Boot 3 upgrade.

```
<plugin>
  <groupId>org.openrewrite.maven</groupId>
  <artifactId>rewrite-maven-plugin</artifactId>
  <version>5.x.x</version>
  <configuration>
    <activeRecipes>
      <recipe>org.openrewrite.java.spring.boot3.UpgradeSpringBoot_3_0</recipe>
    </activeRecipes>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>org.openrewrite.recipe</groupId>
```

```
        <artifactId>rewrite-spring</artifactId>
        <version>5.x.x</version>
      </dependency>
    </dependencies>
</plugin>
```

Slide 17: Before and After: Code Example

Title: A Glimpse of the Magic: Code Transformation

Content:
Here's a simple example of a common change from the javax to jakarta namespace.
Before:

```
package com.example;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
        // ...
    }
}
```

After (after running the recipe):

```
package com.example;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {
    @Override
    public void doGet(HttpServletRequest req, HttpServletResponse resp) throws
```

```
ServletException, IOException {
        // ...
    }
}
```

Note: This is a simplified example. OpenRewrite handles thousands of such changes automatically.

Slide 18: Beyond the Code: Refactoring Build Files

Title: The Full Picture: Handling Dependencies

Content:
OpenRewrite doesn't just refactor Java code; it also intelligently updates build configuration files.

- **Dependency Coordinates:** Updates the groupId and artifactId for Jakarta-compatible versions of dependencies.
- **Property Keys:** Corrects old configuration properties in application.yml and application.properties.
- **Plugin Versions:** Ensures that Maven and Gradle plugins are compatible with the new framework and Java version.

Slide 19: The Roadmap and Community

Title: The Future of OpenRewrite

Content:
OpenRewrite is an active open-source project with a strong community.

- **Growing Language Support:** While focused on Java, the community is building recipes for other languages like Kotlin and Python.
- **AI Integration:** The team is exploring how to use AI to generate custom recipes based on natural language descriptions.
- **Enterprise-Grade Platform:** The creators of OpenRewrite also offer the Moderne platform, which runs recipes at a massive scale for large organizations.

Slide 20: Conclusion & Next Steps

Title: Summary and Q&A

Content:

In summary, OpenRewrite is a game-changer for code modernization.

- It automates large-scale refactorings, saving time and reducing risk.
- It's powered by a comprehensive recipe catalog that is constantly growing.
- It provides a structured, predictable approach to complex migrations.
- The community is vibrant and the tool is continuously evolving.

Next Steps:

- Explore the public recipe catalog on the OpenRewrite website.
- Try the rewrite:dryRun command on one of your own projects.
- Join the OpenRewrite Slack or Discord community.
- Start your journey to a cleaner, more modern codebase!